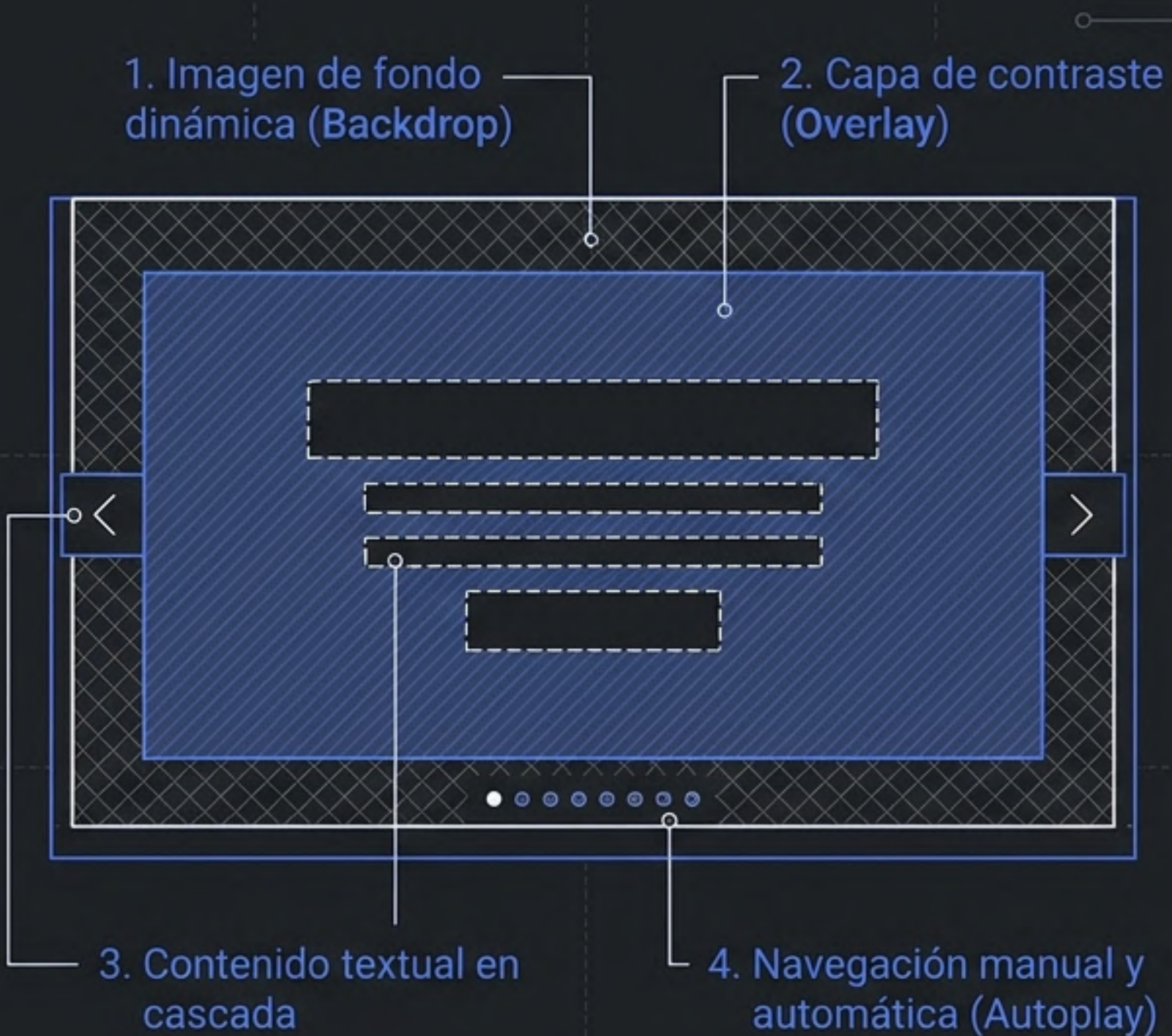


Animaciones Visuales en Vue 3 con Anime.js

Anatomía, estado y reactividad
del componente MyCarousel.vue



La Meta: ¿Qué estamos construyendo?



MyCarousel.vue no es solo una presentación de imágenes. Es un componente rico e interactivo.

- Impulsado por datos (API)
- Organizado en capas visuales (CSS)
- Orquestado por reactividad (Vue 3)

El Motor Core: El flujo de la animación

Una animación de interfaz no es un evento aislado.
Es una coreografía estricta de tres pasos.

1. Estado Reactivo



El usuario cambia el índice (`currentIndex`).
Vue registra la mutación.

2. DOM Actualizado



Vue re-renderiza el HTML.
Usamos `nextTick` para esperar a que termine.

3. Animación Visual



Anime.js toma control de los nuevos elementos y ejecuta el movimiento.

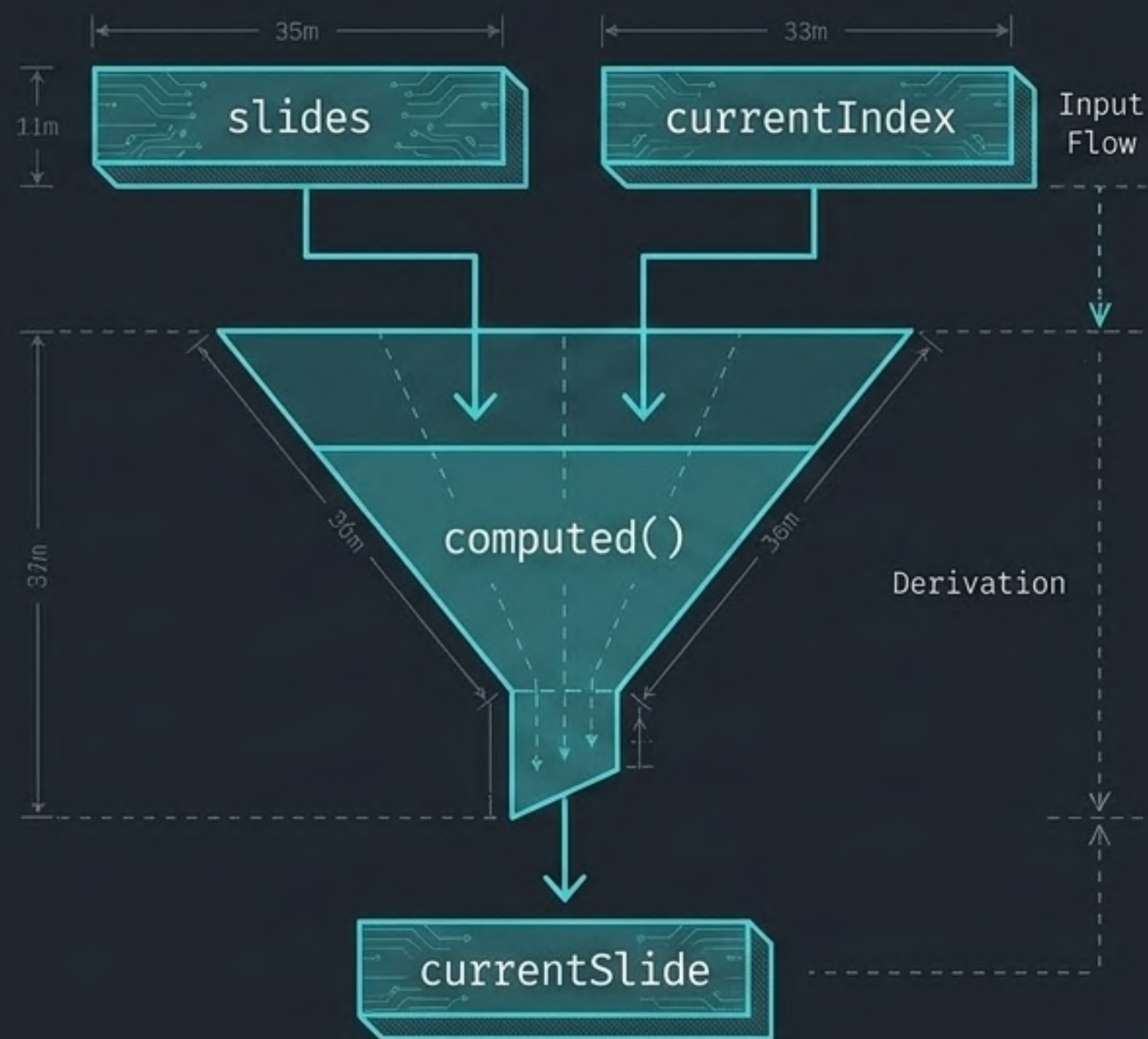
Reactividad Base: El estado del carrusel

Utilizamos `ref` para almacenar los datos que mutarán en el tiempo. El índice es el corazón de la navegación.



Derivando Datos con computed

`computed` centraliza la lógica. Si el índice o el arreglo cambian, calcula automáticamente la nueva diapositiva.



```
import { computed } from 'vue';

const currentSlide = computed(() => {
  // Retorna solo el objeto activo
  return slides.value[currentIndex.value];
});
```



Dato Clave: Evita repetición en el template usando directamente `currentSlide.titulo`.

Interacción: Navegación Circular

El operador módulo (%) permite un ciclo infinito y seguro, devolviendo el índice a 0 al llegar al final.

Visual Math



Code

```
const nextSlide = () => { // ej: 4
  const total = slides.value.length;

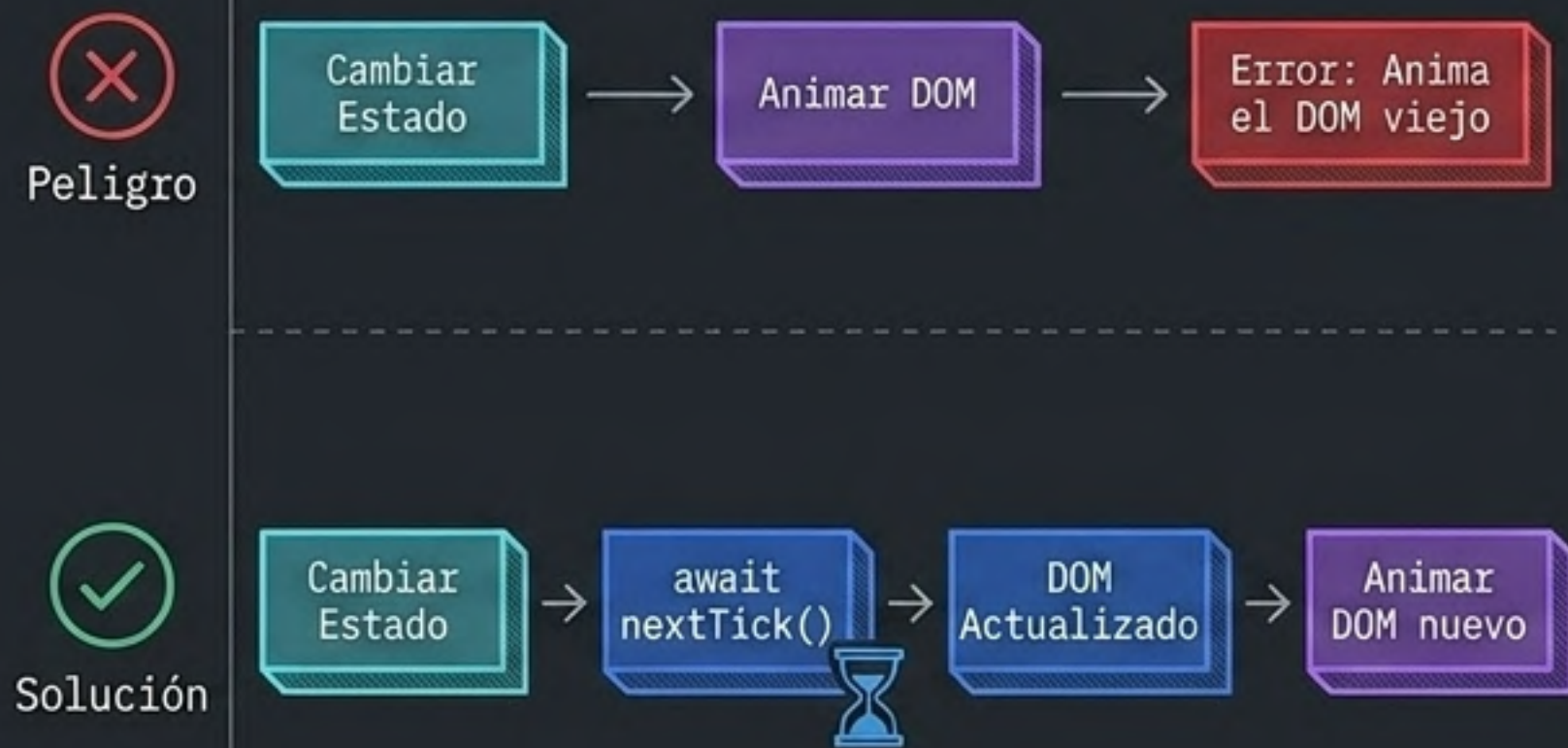
  currentIndex.value = (currentIndex.value + 1) % total;
};
```

Modulo Operator

Circular Indexing

El Puente Crítico: ¿Por qué usar nextTick?

Vue agrupa los cambios. `nextTick` detiene nuestro código hasta que el DOM real refleje el nuevo estado.



```
import { nextTick } from 'vue';

const irA = async (index) => {
  currentIndex.value = index; // 1. Cambia estado
  await nextTick();           // 2. Espera renderizado
  ejecutarAnimaciones();      // 3. Anima el DOM nuevo
};
```


El Motor Visual: Ejecución de Anime.js

La librería necesita saber dos cosas fundamentales: a quién animar (targets) y cómo animarlo.

```
import anime from 'animejs/lib/anime.es.js';
```

```
anime({
```

```
  targets: '.hero-carousel__bg',
```

```
  opacity: [0, 1],
```

```
  duration: 800,
```

```
  easing: 'easeOutQuad'
```

```
});
```

[A quién] Selector CSS

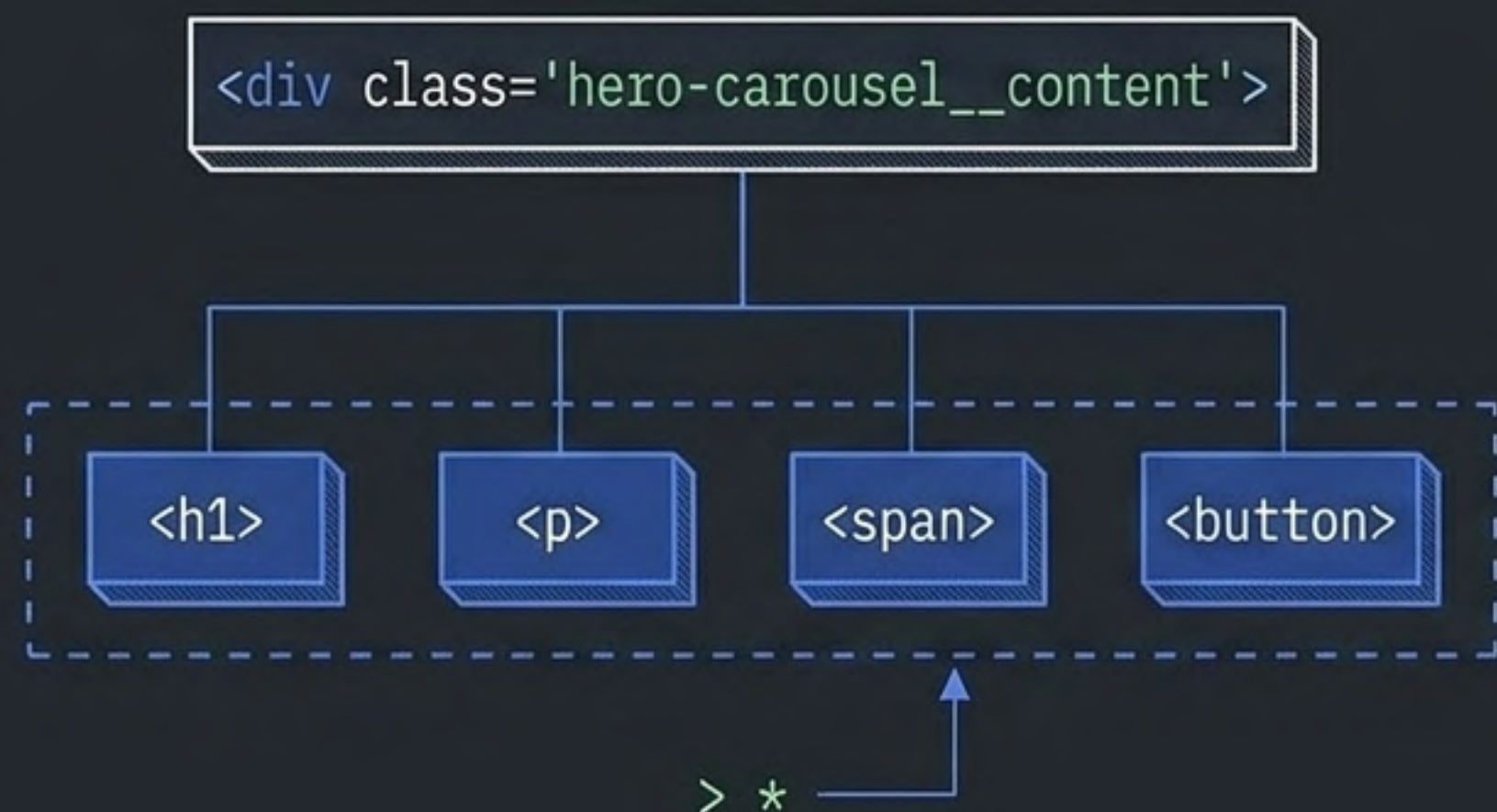
[Qué] De 0 a 100% visible

[Cuánto] 800 milisegundos

[Cómo] Suavizado al final

Múltiples Objetivos (Targets) y Selectores

Podemos apuntar a conjuntos enteros de elementos usando selectores CSS estándar.



```
anime({  
  // Anima todos los textos al mismo tiempo  
  targets: '.hero-carousel__content > *',  
  opacity: [0, 1],  
  translateY: [20, 0] // Sube 20px  
});
```


El Efecto Cascada con stagger

stagger distribuye el inicio de cada animación progresivamente, creando una entrada escalonada y fluida.



```
anime({
  targets: '.hero-carousel__content > *',
  opacity: [0, 1],
  translateY: [20, 0],
  // Empieza en 150ms. Suma 120ms entre cada hijo.
  delay: anime.stagger(120, { start: 150 })
});
```

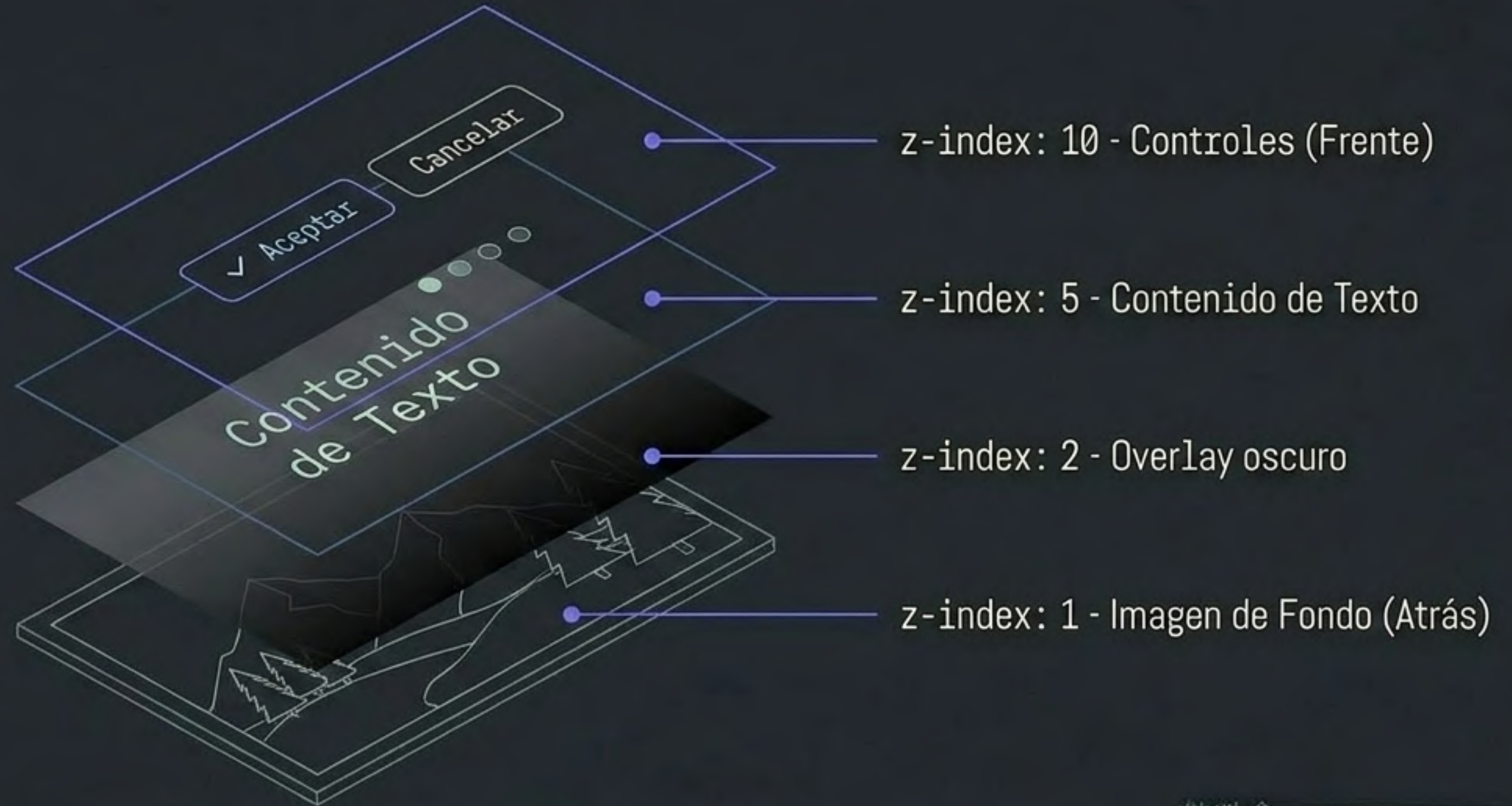

Performance: Propiedades Eficientes

Para evitar caídas de fotogramas, anima solo propiedades que el navegador pueda procesar directamente en la tarjeta gráfica (GPU).

✓ Óptimas (GPU)	✗ Evitar animar (CPU)
 • <code>opacity</code> (Aparecer/Desaparecer) • <code>transform: scale</code> (Acercar/Alejar) • <code>transform: translateY</code> (Mover) No recalculan el layout de la página.	 • <code>width</code> / <code>height</code> • <code>top</code> / <code>left</code> Fuerzan al navegador a re-dibujar toda la página.

Arquitectura Visual: Capas y Overlay

position: absolute y z-index separan responsabilidades. El Overlay garantiza que el texto sea siempre legible.

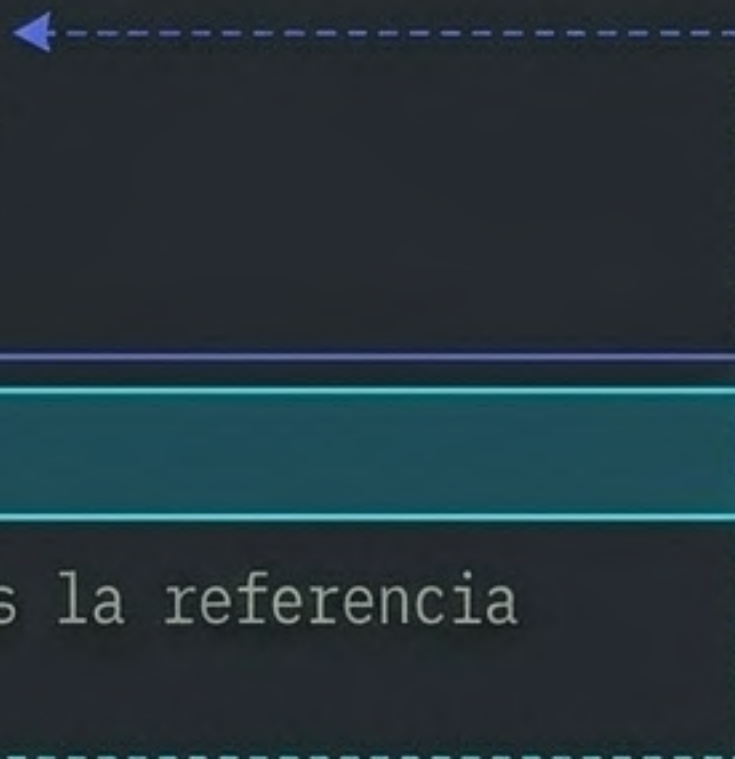


Autoplay Inteligente e Interactivo

Pausamos el carrusel temporalmente cuando el cursor entra al componente para no interrumpir al usuario.

HTML / DOM

```
<div class='carousel' \  
  @mouseenter='pausarAutoplay' \  
  @mouseleave='iniciarAutoplay'>  
  <!-- Contenido -->  
</div>
```



Lógica Vue

```
let intervalo = null; // Guardamos la referencia  
  
const iniciarAutoplay = () => {  
  intervalo = setInterval(nextSlide, 5000);  
};  
const pausarAutoplay = () => clearInterval(intervalo);
```


Previniendo Memory Leaks (Limpieza)

Si el componente es destruido, el temporizador seguirá consumiendo recursos en memoria si no lo limpiamos.



Limpieza y Recolección de Basura

```
import { onBeforeUnmount } from 'vue';

// Hook de ciclo de vida de Vue 3
onBeforeUnmount(() => {
  if (intervalo) {
    clearInterval(intervalo); // Destruye el timer
  }
});
```

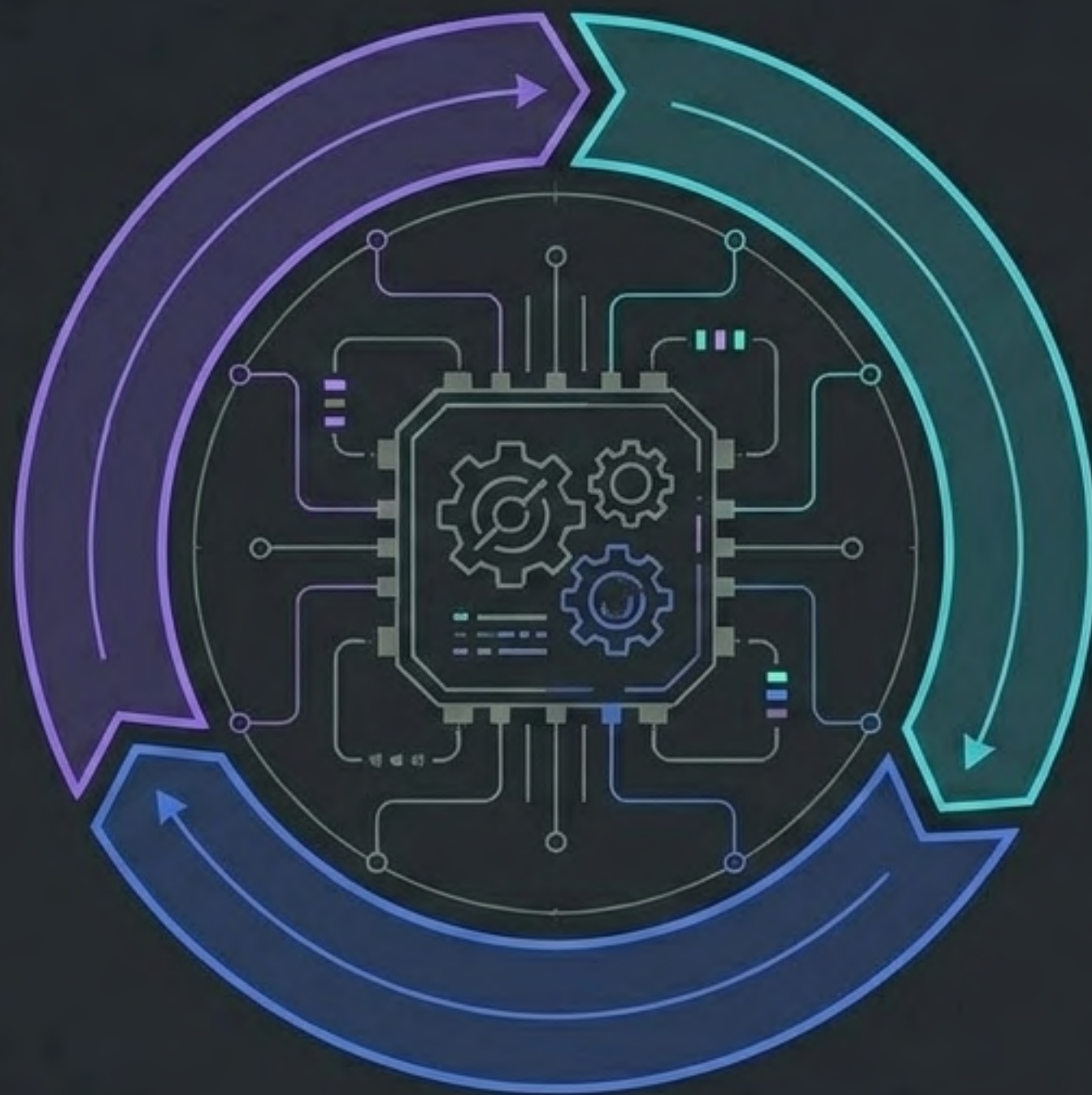
Buena Práctica: Limpiar eventos e intervalos es una obligación profesional en Front-End.

El Motor Completo: Síntesis

Una buena animación no es decoración: comunica estado, jerarquía y mejora la experiencia de usuario.

3. Interpolación

Anime.js usa stagger y GPU para el movimiento fluido.



1. Acción -> Estado

Click o Autoplay muta ref.
Vue deriva computed.

2. Puente de Render

nextTick() sincroniza el
estado lógico con la
pantalla física.